

İSTİSNA YÖNETİMİ

Programlama dünyasında hatalar kaçınılmazdır. Ancak bir yazılımcı için önemli olan hatanın oluşması değil, bu hatanın yönetilebilir olmasıdır. Programın **çalışma anında (runtime)** karşılaştığı beklenmedik durumlara **İstisna (exception)** denir. İstisna yönetimi, programın bu hatalar karşısında tamamen çökmesini engellemek ve kullanıcıya anlamlı geri bildirimler vererek sistemin güvenli bir duruma (**safe state**) geçmesini sağlamak için kullanılır.

Kitabımızın başında belirttiğimiz "nelerin yapılacağını değil, işin nasıl yapılacağını" bilme ilkesi burada devreye girer. **İstisna yönetimi (exception handling)**, sadece hataları gizlemek için değil, sistemin sınırlarını belirlemek için kullanılır.

try-except Bloğu

Python'da istisnalar **try-except** mekanizması ile kontrol edilir. **try** bloğu, hata potansiyeli taşıyan, riskli kodların yazıldığı bölümdür. **except** bloğu ise eğer **try** bloğu içinde bir hata oluşursa, programın akışı anında bu bloğa sıçrar. Söz dizimi aşağıda belirtilmiştir;

```
try:
    # Riskli kodlar bu blokta yazılır
    sayi = int(input("Bir sayı giriniz: "))
    sonuc = 100 / sayi
except ValueError:
    # Veri tipi uyumsuzluğunda bu blok çalışır
    print("Hata: Lütfen geçerli bir tam sayı giriniz!")
except ZeroDivisionError:
    # Sıfıra bölünme hatasında bu blok çalışır
    print("Hata: Bir sayıyı sıfıra bölemezsiniz!")
```

Bazı durumlarda **else** ve **finally** bloğu da kullanılır.

- **else** Bloğu: Eğer **try** bloğunda hiçbir hata oluşmazsa çalışır. İşlemin başarıyla tamamlandığını teyit etmek için kullanılır.
- **finally** Bloğu: Hata oluşsa da oluşmasa da mutlaka çalışır. Özellikle dosya kapatma, veri tabanı bağlantısı kesme veya donanım kaynaklarını serbest bırakma gibi "temizlik" işlemleri için hayati önem taşır.

Python dilinde diğer nesne yönelimli dillerde olduğu gibi tüm istisnalar aslında birer nesnedir (object) ve **BaseException** sınıfından türetilmiştir. Hataları yakalarken "özelden genele" doğru bir sıra izlenmelidir.

```
try:
    dosya = open("veriler.txt", "r")
    # ... işlemler ...
except FileNotFoundError:
    print("Dosya bulunamadı!")
except Exception as e:
    # Tüm hataları yakalayan genel sınıf
    print(f"Beklenmedik bir hata oluştu: {e}")
finally:
    print("Sistem kaynakları kontrol ediliyor...")
```

Neden try-except Kullanmalıyız?

- **Süreklilik:** Programın küçük bir hata yüzünden tamamen durmasını engeller.
- **Güvenlik:** Kullanıcıya sistemin iç yapısını **ele veren (traceback)** karmaşık hata mesajları yerine, filtrelenmiş ve güvenli mesajlar gösterilmesini sağlar.
- **Hata Ayıklama (debug):** Hatanın nerede ve neden oluştuğuna dair **iz kaydı (log)** tutulmasını kolaylaştırır.

Python'da her hata aslında bize sistemin neresinde bir mantık veya kaynak sorunu olduğunu söyler. Aşağıdaki tablo, geliştirme sürecinde en sık karşılaşılabilecek istisnaları ve bunların bilgisayar mimarisindeki karşılıklarını özetlemektedir;

<i>İstisna (Exception)</i>	<i>Ne Anlama Gelir?</i>	<i>Mühendisçe Yaklaşım</i>
SyntaxError	Yazım hatası. Python'un dil kurallarına aykırı kod yazılmıştır.	Kod daha derleme/yorumlanma aşamasına geçememiştir.
IndentationError	Girintileme hatası. Blok yapısı bozulmuştur.	Python'un kapsam (scope) belirleme kuralı ihlal edilmiştir.
NameError	Tanımlanmamış bir değişken veya fonksiyon adı kullanılmıştır.	Bellekte (RAM) karşılığı olmayan bir isme referans verilmiştir.
TypeError	Bir veri tipi, desteklemediği bir işleme tabi tutulmuştur.	Nesne arayüzü (interface) ile yapılan işlem uyumsuzdur.
ValueError	Veri tipi doğru olsa da içeriği (değeri) işlem için uygun değildir.	Fonksiyonun kabul edilebilir parametre sınırları dışına çıkmıştır.
ZeroDivisionError	Bir sayı sıfıra bölünmeye çalışılmıştır.	Matematiksel bir tanımsızlık ve işlemci seviyesinde bir hata durumudur.
IndexError	Bir listenin veya dizinin sınırları dışında bir indise erişilmeye çalışılmıştır.	Bellekte tahsis edilen alanın (buffer) dışına taşma girişimidir.
KeyError	Sözlükte (dict) bulunmayan bir anahtar (key) çağırılmıştır.	Hash tablosunda aranan referans bulunamamıştır.
FileNotFoundError	Erişilmek istenen dosya belirtilen yolda mevcut değildir.	Dosya sistemi (File System) üzerinde bir girdi/çıkı (I/O) hatasıdır.
AttributeError	Bir nesneye, sahip olmadığı bir özellik veya metot ile erişilmeye çalışılmıştır.	Sınıf hiyerarsisi veya nesne yapısı yanlış kurgulanmıştır.

Tablo 20.İstisna Türleri

Python bir hata verdiğinde, terminalde **geriye doğru izlenen (traceback)** bir çıktı üretir. Son fonksiyon çağrıları için bu bir **yığın izidir (stack trace)**. En alt satır hatanın türünü ve nedenini söyler. Üstteki satırlar ise hatanın kodun hangi katmanlarından geçerek (fonksiyon çağrı yığını) buraya ulaştığını gösterir. Hata ayıklarken her zaman en alttan başlayıp yukarı doğru "yığın boyunca" takip yapmalısınız.

İstisnai Duruma Düşme

Şu ana kadar istisnaların Python yorumlayıcısı tarafından otomatik olarak oluşturulup sanal bir istisna havuzuna atılmasını (sıfıra bölme, hatalı tip vb.) ve bizim bunları **except** ile **yakalamamızı** işledik. Ancak profesyonel bir yazılım mimarisinde, bazen kod teknik olarak doğru olsa da mantıksal olarak hatalı bir durum oluşabilir. Bu durumda mühendis, hatayı kendisi tetiklemeli yani hataya ait istisna nesnesini kendisi imal edip istisna havuzuna **atmalıdır (raise)**. İşte bu işleme **istisnai duruma düşme (raising an exception)** adı verilir.

Kendi istisna nesnelerimizi aşağıdaki durumlarda kullanabiliriz;

1. İş Kurallarını Denetlemek: Örneğin, bir bankacılık uygulamasında bakiyenin altına düşen bir çekim işlemi teknik bir hata değildir ama mantıksal bir hatadır.
2. Hata Zinciri Oluşturmak: Alt katmanlarda oluşan bir hatayı yakalayıp, daha üst katmana daha anlamlı bir mesajla iletme için.
3. Zorunlu Alan Kontrolü: Fonksiyonun çalışması için kritik olan verilerin eksikliği durumunda süreci durdurmak için.

Kullanıcı tanımlı bir istisna nesnesi **raise** talimatı ile verilir. Sözdizimi aşağıda verilmiştir;

```
raise <İstisnaTürü>("Hata Mesajı")
```

Buradaki istisna türü Tablo-20’de verilen istisna sınıflarından biri olabilir. Örnek olarak; bir mühendislik projesinde, bir sıcaklık sensöründen gelen verinin fiziksel sınırlar (-50°C ile +150°C) dışında olması durumunda sistemin durdurulması gerektiğini varsayalım. Buna ilişkin kod aşağıda verilmiştir;

```
def sicaklik_kontrol_et(deger):
    if deger < -50 or deger > 150:
        # Teknik bir hata yok ama fiziksel bir imkansızlık/hata var
        raise ValueError(f"Geçersiz sıcaklık verisi: {deger}. Sensör arızalı olabilir!")
    print(f"Sıcaklık normal: {deger}")

try:
    sicaklik_kontrol_et(200)
except ValueError as e:
    print(f"Sistem Durduruldu: {e}")
```

Bazen bir hatayı yakalayıp, üzerine kendi açıklamamızı ekleyerek yeni bir istisna nesnesi imal ederiz. Buna **hata zincirleme** (**exception chaining**) denir.

```
try:
    # Veritabanına bağlanma denemesi
    baglanti_kur()
except ConnectionError as orijinal_hata:
    # Hatayı yakalayıp daha üst seviye bir mesajla fırlatıyoruz
    raise RuntimeError("Veritabanı erişimi başarısız, sistem başlatılamıyor.") from
    orijinal_hata
```

Kendi İstisna Sınıflarımızı Oluşturma

Büyük projelerde Python’un standart hata isimleri (ValueError, TypeError vb.) yetersiz kalabilir. Bu durumda, kitabımızın **kalıtım** (**Inheritance**) prensibini kullanarak kendi hata/istisna sınıflarımızı (**custom exception**) oluşturabiliriz.

```
class SensorArizasi(Exception):
    """Sensör donanım hataları için özel istisna sınıfı"""
    pass

# Kullanımı:
raise SensorArizasi("Donanımdan sinyal alınamıyor!")

class YasHatasiError(ValueError):
    """Geçersiz yaş değeri hatası"""
    def __init__(self, yas, mesaj="Yaş 0-150 arası olmalı"):
        self.yas = yas
        self.mesaj = f"{mesaj}. Girilen: {yas}"
        super().__init__(self.mesaj)
```